

目录

1	项目目标与总体思路	1
1.1	评价目标与边界	2
1.2	工程目录结构	2
2	公共基础设施：分层 IR 与验证体系	3
2.1	前端与语义分析	3
2.2	canonical IR	4
2.3	lower IR	4
2.4	ValueSSA 与 MemorySSA	4
2.5	分层 IR 示例	5
3	-riscv 模式：基础编译与 RISC-V 输出	6
3.1	设计目标	6
3.2	后端分层	7
3.3	后端落地示例	7
3.4	效果	8
4	-perf 模式：优化、SSA pass 与分配实验	8
4.1	优化 workflow	8
4.2	多层优化主线	9
4.3	family-aware 的 SSA 分配器	9
4.4	效果与边界	15
5	-extension 模式：语言扩展与 callable-object IR	15
5.1	扩展特性概览	16
5.2	defer / fndefer / capdefer	16
5.3	pair、struct 与 aggregate	19
5.4	函数值与闭包	21
6	端到端翻译案例	27
6.1	主 showcase: aggregate + returned closure	27
6.2	aggregate lowering 关键片段	28
6.3	callable-object 关键片段	29
6.4	lower IR 与 RISC-V 关键片段	29
6.5	defer / capdefer / fndefer 展示	30
7	严格返回路径审计	31
7.1	为什么需要单独做这个检查	31
7.2	实现思路	31
7.3	循环与不可达 return	32
7.4	边界与测试价值	32

8 Vibe 工作流与质量保障	32
8.1 AI agent 协作设定	33
8.2 测试矩阵	33
8.3 Verifier 与 dump	34
8.4 全量回归	34
9 经验教训	34
9.1 分层比局部技巧更重要	34
9.2 语言扩展必须保守打开	34
9.3 性能优化必须讲证据	34
9.4 测试不是收尾，是设计的一部分	35
10 总结	35
PS	35
A Showcase 源程序与复现命令	35
A.1 主 showcase 源程序	35
A.2 defer showcase 源程序	36
A.3 复现命令	36
B 项目规模统计	36
C 运行与测试工具链	37
C.1 构建编译器	37
C.2 编译一个源程序	37
C.3 从汇编链接并运行	38
C.4 dump 中间层	38
C.5 测试矩阵	38
C.6 extension runtime 与 SysY suite	39
C.7 性能与诊断工具	39
C.8 报告编译	40
D 进一步阅读路线	40
D.1 项目总览与工作记忆	40
D.2 前端、语义与 strict return	40
D.3 IR、lower IR 与可观察 dump	40
D.4 SSA、MemorySSA 与优化	41
D.5 family-aware allocator	41
D.6 后端与 RISC-V 输出	41
D.7 -extension 语言扩展	42
D.8 测试体系	43

基于 SysY 的多模式编译器设计与实现报告

(朱宏泽 / 2400012623)

项目仓库: [glazecats/koopu-compiler](https://github.com/glazecats/koopu-compiler)

2026 年 6 月 10 日

摘要

本项目实现了一个以 SysY/类 C 语言为基础的课程编译器。与只追求单一路径“能编译”不同,本项目将编译流程拆分为三个面向不同目标的模式: `-riscv` 面向基础正确编译与 RISC-V 汇编输出, `-perf` 面向性能优化与分配实验, `-extension` 面向语言特性扩展。三条模式共享前端、语义分析、分层 IR、SSA 与后端基础设施,但各自采用不同的保守边界和测试策略。项目的核心特色包括 canonical IR 与 lower IR 的分层设计、ValueSSA/MemorySSA 风格的中端结构、family-aware 的 SSA 分配实验,以及以函数值、闭包和 callable-object IR 为代表的语言扩展主线。开发过程中采用快速迭代的 vibe coding 工作流,但每个迭代都由 verifier、dump regression、runtime regression 和全量测试闭环约束;也就是说,快速实现必须始终服从可验证的工程纪律。

1 项目目标与总体思路

本项目的目标是实现一条完整、可验证、可扩展的编译器主线。输入语言以 SysY 风格的类 C 语言为基础,输出方向以 RISC-V 汇编为主,同时保留面向中间表示观察、优化实验和语言扩展的多条工程路径。

从整体结构看,编译流程可以概括为:

```
source → lexer → parser → AST → semantic
      → canonical IR → lower IR → ValueSSA / MemorySSA
      → machine backend → RISC-V asm
```

这个项目最重要的设计选择,是没有把所有能力塞进一条难以控制的“大管线”,而是按目标拆成三个模式:

1. `-riscv`: 面向课程基础目标,强调正确翻译、稳定输出和后端连通。
2. `-perf`: 面向性能目标,在正确性闭环下进行优化和寄存器分配实验。
3. `-extension`: 面向语言扩展,用保守 pipeline 接纳更复杂的语义特性。

三种模式不是三套互不相干的编译器,而是同一个工程在不同约束下的三种使用方式。基础输出、性能优化和语言扩展放在一起,使项目更接近真实编译器工程,而不只是完成单一评测目标的最小实现。

1.1 评价目标与边界

除了三个主模式外，driver 还提供了一个可叠加的严格检查开关：

```
--enforce-all-paths-return-check
```

它不是第四种编译模式，而是放在 `-riscv`、`-perf`、`-extension` 前面的语义审计选项，用于更严格地检查非 `void` 函数是否在所有控制流路径上返回。它也会提前拦截一类静态可判定的非终止/不可达返回问题：例如明显会卡在 `while(1)` 或 `for(;;)` 中的路径，不能靠循环后面不可达的 `return` 来伪装成“所有路径都返回”。默认编译路径为了兼容课程提交和较宽松的 SysY 测试习惯，会有意绕开这一严格 gate；当需要审计控制流返回性质时，再显式打开该选项。

因此，本项目的效果评价也按目标拆开：

- `-riscv`: 重点看基础程序能否稳定通过语义检查、lowering 和后端，最终生成可汇编的 RISC-V 文本；
- `-perf`: 重点看优化是否带来真实运行收益，并通过 A/B、compile-time 和正确性回归共同决定是否保留；
- `-extension`: 重点看复杂语言特性能否被诚实地分阶段支持，支持的形状要有 runtime 或 dump regression，暂不支持的形状要明确 gate；
- `strict return-check`: 重点看控制流返回分析是否足够保守、可解释，并避免把合法程序误判为缺失返回。

这也说明了项目的一个 non-goal: 它并不试图一次性实现完整工业级 C 编译器或完整一等函数运行时，而是在课程编译目标之上，逐步把 IR、SSA、后端和扩展语义做成可验证、可继续生长的工程。换句话说，暂未开放的能力应当由明确的 gate 拒绝，而不是在边界不清的位置生成不可靠的中间表示。

1.2 工程目录结构

从仓库结构上看，项目并不是把所有实现压在一个目录里，而是按编译阶段和工程职责拆分：

- `src/` 与 `include/`: 编译器实现与公共头文件，包括前端、IR、SSA、后端和 runtime/observe 相关模块；
- `tests/`: 按 lexer、parser、semantic、IR、lower IR、SSA、machine、driver、extension runtime 等层次组织的回归测试；
- `docs/`: 工程记忆、下一步计划、阶段设计文档和优化/语言特性路线；
- `lesson/`: 由 review/lesson agent 精炼出的项目说明材料，用于给报告和后续维护提供压缩上下文；
- `tools/`: dump、profile、diagnostic 等辅助工具，用于观察中间阶段；
- `reports/`: 课程报告源文件与生成的 PDF。

这样的目录结构也对应了项目的协作方式：实现、测试、设计记录、精炼 lesson 和课程报告各自有位置，不需要把所有上下文集中到单一总文档中。

2 公共基础设施：分层 IR 与验证体系

2.1 前端与语义分析

前端部分包括词法分析、语法分析、AST 构造和语义分析。词法分析负责将字符流切分成 token；语法分析将 token 组织成声明、语句和表达式；AST 作为 source-near 的结构化表示，保留后续语义检查和 lowering 所需的信息。语义分析则负责名字解析、作用域、类型检查、函数签名、控制流返回约束等规则。

项目后期加入了不少扩展特性，例如 `defer`、函数值、闭包、结构体、保守浮点支持等，因此语义分析不只负责基础名字解析。它还承担了模式边界控制：哪些语法只在 `-extension` 下开放，哪些组合仍然应该被拒绝，必须在语义层给出明确诊断。

虽然这些前端步骤在课程编译器中相对标准，但它们仍然是后续 IR 正确性的入口。例如下面这段源码：

```
int add(int x, int y) {
    return x + y;
}
```

词法分析会先把字符流切成关键字、标识符、标点和运算符：

```
INT IDENT(add) LPAREN INT IDENT(x) COMMA INT IDENT(y) RPAREN
LBRACE RETURN IDENT(x) PLUS IDENT(y) SEMICOLON RBRACE
```

语法分析和 AST 构造再把这些 token 组织成“函数定义、参数列表、返回语句、二元表达式”这样的树形结构：

```
FunctionDef name=add return=int
  Param x:int
  Param y:int
  Body
    Return
      Binary(+)
        Identifier(x)
        Identifier(y)
```

语义分析则在 AST 上补充更“看不见”的约束：`x` 和 `y` 是否在当前作用域可见，`x + y` 的类型是否为 `int`，返回表达式是否匹配函数返回类型，以及非 `void` 函数是否满足当前模式要求的返回路径规则。换句话说，lexer/parser/AST 给出程序的结构，semantic 判断该结构是否满足语言规则。

语义层还负责把一些错误尽早挡在 IR 之前。例如：

```
int bad(int x) {
    if (x > 0) return x;
}
```

在默认课程兼容路径下，这类程序可以按较宽松规则处理；但打开严格返回检查开关后，semantic 会指出非 void 函数存在未返回路径。类似地，若程序在非 `-extension` 模式下使用闭包或函数值，或者在 `-extension` 下写出尚未开放的函数值组合，也会在语义阶段被明确拒绝，而不是生成语义契约已经不可靠的中间表示。这个前端的目标不是增加表面复杂度，而是把模式边界和错误诊断尽量前置：通过的程序可以交给后续 verifier 和 lowering，未通过的程序也要尽量说明拒绝原因。

2.2 canonical IR

canonical IR 是语义分析之后的第一层稳定中间表示。它是 block-based、非 SSA 的 IR，保留较多接近源码的控制流结构，适合做早期规范化和轻量清理。

这一层的主要职责是：

- 将 AST 中的表达式、语句、控制流转成统一 IR；
- 用基本块和 terminator 显式表达控制流；
- 通过 verifier 检查 block、temp、local、call、return 等结构契约；
- 运行常量折叠、copy propagation、CFG cleanup 等轻量 pass。

2.3 lower IR

lower IR 的核心思想是显式区分 value 和 storage。canonical IR 中的 local/global 在 lower IR 中不再随意作为普通 value 出现，而是通过 `load_*` 和 `store_*` 指令访问。

这一层的意义在于：它给后续 SSA construction 提供了更干净的输入。ValueSSA 只需要处理 value flow，而不必同时承担完整 memory SSA 的语义责任。这是一种很“工程化”的分层：先把边界说清楚，再往后做复杂分析。

2.4 ValueSSA 与 MemorySSA

ValueSSA 将 lower IR 的值流转换为 SSA 形式，引入显式的 SSA value、def-use 关系和 phi 节点。这里的 SSA value 就是静态单赋值意义上的值名：每个 `ssa.*` 只定义一次，后续使用都通过 def-use 关系追踪。它主要处理值流，而不是完整内存状态。MemorySSA 则作为后续补充，用于表达和优化内存版本、load/store 关系和内存依赖。

这种拆分使得项目可以分别推进：

- ValueSSA 上的常量折叠、冗余二元表达式消除、CFG 简化、dead value DCE；
- MemorySSA 上的 load forwarding、store cleanup、memory CSE；
- allocator 和 machine backend 所需的 liveness、interference、allocation prep。

2.5 分层 IR 示例

下面用一个很小的条件表达式说明三层 IR 的差异。源码本身并不复杂，但它已经包含控制流汇合：

```
int choose(int a, int b, int c) {
    return c ? a + 1 : b + 2;
}
```

在 canonical IR 中，条件表达式被拆成显式基本块。两个分支分别计算结果，并汇合到同一个临时值 tmp.0：

```
func choose(a.0, b.1, c.2) {
    bb.0:
        br c.2, bb.1, bb.2
    bb.1:
        tmp.1 = add a.0, 1
        tmp.0 = mov tmp.1
        jmp bb.3
    bb.2:
        tmp.2 = add b.1, 2
        tmp.0 = mov tmp.2
        jmp bb.3
    bb.3:
        ret tmp.0
}
```

lower IR 则把 storage 边界显式化。参数和局部变量不再被当作普通值随意消费，而是先通过 load_local 读出；这里没有局部变量写回，所以不会出现 store_local：

```
func choose(a.0, b.1, c.2) {
    bb.0:
        tmp.3 = load_local c.2
        br tmp.3, bb.1, bb.2
    bb.1:
        tmp.4 = load_local a.0
        tmp.1 = add tmp.4, 1
        tmp.0 = mov tmp.1
        jmp bb.3
    bb.2:
        tmp.5 = load_local b.1
        tmp.2 = add tmp.5, 2
        tmp.0 = mov tmp.2
        jmp bb.3
    bb.3:
        ret tmp.0
}
```

ValueSSA 再把 value flow 改写成静态单赋值形式。两个分支中产生的值分别是 `ssa.3` 和 `ssa.6`，汇合块用 `phi` 根据控制流前驱选择正确来源：

```
func choose(a.0, b.1, c.2) {
  bb.0:
    ssa.0 = load_local c.2
    br ssa.0, bb.1, bb.2
  bb.1:
    ssa.1 = load_local a.0
    ssa.2 = add ssa.1, 1
    ssa.3 = mov ssa.2
    jmp bb.3
  bb.2:
    ssa.4 = load_local b.1
    ssa.5 = add ssa.4, 2
    ssa.6 = mov ssa.5
    jmp bb.3
  bb.3:
    ssa.7 = phi [bb.1: ssa.3], [bb.2: ssa.6]
    ret ssa.7
}
```

这个例子很小，但能看出分层的核心：canonical IR 方便从 AST 生成和做早期清理；lower IR 把 value/storage 说清楚；ValueSSA 再用静态单赋值和 `phi` 把跨控制流的值选择显式化。每层只解决本层的结构问题，可以减少后续阶段的隐式假设和调试成本。

3 -riscv 模式：基础编译与 RISC-V 输出

`-riscv` 模式是项目的基础交付模式。它面向课程编译器的核心目标：将 SysY/类 C 程序正确编译为 RISC-V 汇编，并尽量保持输出可被标准工具链接受。

3.1 设计目标

`-riscv` 模式强调三个关键词：

1. 正确性：源程序语义必须被保守、稳定地翻译。
2. 可汇编性：输出文本应能通过 RISC-V 汇编器/链接器检查。
3. 可维护性：后端每一层都有自己的 artifact、summary 和测试入口。

在该模式下，语言能力主要围绕课程基础需求展开，不默认接受所有 `-extension` 特性。这避免了扩展语义对课程基础路径造成额外风险。

3.2 后端分层

后端主线大致可以拆成：

```
machine_ir → machine_select → machine_layout
           → machine_emit → machine_encode → RISC-V text
```

其中，machine IR 承接 ValueSSA/allocator 之后的 machine-facing 表示；machine select 将较抽象的操作选择成更接近目标机器的 selected op；layout/emit/encode 则逐步处理块布局、标签、目标 profile 和最终 RISC-V 文本输出。

除了这条主交付路径外，项目还建立了 object/runtime/observe 相关的支撑层，例如 bytes、object、reloc、ELF、image、runtime、trace、journal 等。它们在本文中不作为独立主线展开，但很能说明后端的工程取向：后端不是一个把字符串拼成汇编的“printf 汇编”文件，而是一组带 artifact、summary、verifier 和测试入口的阶段。这样做的好处是，某一层出问题时可以从中间产物定位；某一层需要替换时，也不必把整个后端拆成一地零件。

3.3 后端落地示例

沿用上一节的 choose 例子：

```
int choose(int a, int b, int c) {
    return c ? a + 1 : b + 2;
}
```

ValueSSA 中的 phi 表达了“从哪个分支来的值”。进入后端后，这个例子会被进一步整理成两个直接返回的分支，因此 machine-facing 形状已经不再保留显式汇合块：

```
machine_select
function choose params=3 locals=3 spills=0
  bb.0:
    reg.0 = load_local local.2
    br reg.0, bb.1, bb.2
  bb.1:
    reg.0 = load_local local.0
    reg.0 = alui.0 reg.0, 1
    ret reg.0
  bb.2:
    reg.0 = load_local local.1
    reg.0 = alui.0 reg.0, 2
    ret reg.0
```

最终输出的 RISC-V 文本则体现为参数落栈、条件分支、两个返回路径和栈帧恢复：

```
choose:
  addi sp, sp, -16
  sw a0, 0(sp)
  sw a1, 4(sp)
  sw a2, 8(sp)
```

```
.Lchoose_0:
    lw a0, 8(sp)
    bne a0, zero, .Lchoose_2
.Lchoose_1:
    lw a0, 4(sp)
    addi a0, a0, 2
    addi sp, sp, 16
    ret
.Lchoose_2:
    lw a0, 0(sp)
    addi a0, a0, 1
    addi sp, sp, 16
    ret
```

这个片段展示了 `-riscv` 模式的核心效果：前面 IR 中较抽象的控制流和值选择，在后端被转成目标机器可接受的标签、分支和寄存器/栈操作。中间形状可以变化，但每一步都有 dump 可以观察，因此比只检查最终汇编文本更容易定位问题。

3.4 效果

`-riscv` 模式最终希望达到的效果是：

- 能完成基础 SysY 程序的前端检查、中端 lowering 和后端输出；
- 生成 assembler-accepted 的 RISC-V preview asm；
- 对函数调用、全局变量、数组/间接内存、控制流、返回值等基础语义保持正确；
- 在后端 cleanup 中避免危险 peephole，例如跨 call 错误复用 caller-clobbered register。

4 -perf 模式：优化、SSA pass 与分配实验

`-perf` 模式面向性能优化。与 `-riscv` 相比，它更关注运行时间、热点程序和优化收益，但仍然以正确性为前提。

4.1 优化 workflow

项目的性能优化 workflow 遵循两个紧凑原则：优化候选先来自热点或结构证据，是否保留再由 A/B、正确性和编译时间共同决定。也就是说，优化不是看到汇编里某个形状不理想就直接改，而是结合：

- 源程序的循环、递归和调用结构；
- 生成汇编中的重复计算、地址链、调用开销；
- 实际运行时间和热点 witness；

- 编译时间变化和正确性回归风险。

这种流程在课程项目里略显“重”，但很有价值。因为优化很容易产生误导性信号：静态代码形状变短，并不必然意味着运行时间变短，更不必然意味着语义仍然正确。因此优化候选必须经过运行结果、性能数据和回归测试共同确认。

4.2 多层优化主线

-perf 的主要优化发生在 ValueSSA 及其相邻层，但并不只发生在 SSA。项目更接近一组分层的优化实验场：前面的 IR 层适合做语义更清楚的规范化，中间 SSA 层适合做 def-use 和控制流优化，后端层则适合做目标相关的局部清理。粗略分布如下：

- canonical IR: 早期 constant folding、copy propagation、CFG cleanup、死临时值清理；
- lower IR: value/storage boundary normalization, 为后续 SSA 和 memory analysis 减少歧义；
- ValueSSA: trivial value simplification、algebraic identity simplification、constant folding、redundant binary elimination、CFG simplification、dead value DCE、tiny helper inlining、LICM/induction/address-carrier 相关 hotspot pass；
- MemorySSA: local/global load forwarding、redundant/dead store cleanup、local/global slot scalar-replacement 风格实验；
- machine IR / machine select: slot-aware cleanup、跨块 alias/copy propagation、基于 live-out 的 dead-def elimination、call barrier 下的 register/spill-slot 区别处理；
- final RISC-V text: 少量 target-specific peephole, 例如 tail-return、repeated-address pattern、stack store/reload 等，但这层必须最谨慎。

这些 pass 的共同特点是：尽量把优化放在语义信息还足够丰富的层上，优先做可验证的小步变换；最终文本 peephole 只处理很局部、目标相关、并且有回归保护的形状。经验上，越靠近最终汇编，可用的语义信息越少，因此文本级优化应当限制在非常具体、可回归验证的模式上。

4.3 family-aware 的 SSA 分配器

-perf 模式中另一个有特色的部分是 ValueSSA allocator。它的基本骨架仍然接近传统图染色分配器：先根据 liveness 建 interference graph，再把低压力节点 simplify/remove 到一个栈式顺序里，最后按逆序 select color；如果某个 value 在逆序染色时没有可用颜色，就进入 spill / rewrite 路线。

项目里的实现没有推翻这个骨架，而是在骨架上加了一层 family-aware policy。朴素实现通常把每个 SSA value 作为相对独立的分配对象处理，逆序染色时能分配则分配，不能分配则 spill；这里会额外问：哪些 value 来自 mov/phi/copy affinity，应该尽量被视为同一组？如果它们想共色，当前 blocker 是不是可以被安全挪开？如果某个值先被当成 spill，后面有没有机会 recover？

更准确的主线可以概括为：

SSA values

```

-> interference / liveness / affinity
-> coalesce pair analysis
-> coalesced family / root
-> family-aware simplify / freeze / spill plan
-> reverse select coloring
    with preferred-color / targeted-eviction / retry-recovery
-> spill slot / SSA rewrite / canonicalize / layout report

```

这个流程包含多个阶段。第一步 liveness/interference 先回答“哪些 SSA value 的生命周期重叠，因此不能放在同一个寄存器颜色里”；affinity/coalesce analysis 再从 mov、phi 和 copy-like value flow 里找出“哪些值最好能共色”。随后 planner 并不是直接染色，而是先根据 pressure、move 关系和 spill 风险生成 removal plan；真正的颜色选择发生在 reverse select coloring 阶段。最后，如果确实 spill，就进入 spill-slot placement、SSA rewrite 和后续 canonicalize。换句话说，报告里的 allocator dump 是一组阶段化诊断信息，包括关键决策点的状态快照和 allocation trace。

其中，family 表示当前被视为同一 coalesced class 的一组值，root 是 family 的代表。当前实现还不是教材里那种完全 merged-node 的 Briggs/Chaitin allocator：最终结果仍然落回每个 ssa.N 的 color/spill placement；但 planner 和 selector 已经会利用 family 信息来调整过程。

这个 family 更接近一种 soft coalescing 视角。实现上先用 union-find 风格的 root 和 class-size 数组记录哪些 value 属于同一组，但不会把整组立刻压成一个真正的 super-node。remove 时仍然是一项一项移出，例如先移出 ssa.3，再移出 ssa.1；即使 root 对应的 value 先被移走，root 也仍然作为 family id 存在。因此实现里会把三件事分开维护：family identity、当前还 active 的 family 成员数，以及 reverse select 时已经染色的 family 成员。plan 完成后还会把动态变化后的 family 写入：

```

final_runtime_coalesce_roots
final_runtime_coalesce_class_sizes

```

select 阶段再通过这些 root/class 信息寻找 family preferred color。

实现上比较关键的概念不是单纯拿来展示，而是会直接进入 allocator 的决策：

- **degree / effective_degree**: planner 用它判断寄存器压力。原始 degree 来自 interference graph；effective_degree 会扣掉 family 内部已经可解释的压力，从而避免把“本来希望共色的一组值”误判成更难分配；
- **move_related / effective_move_related**: planner 和 move-family worklist 用它决定是继续尝试 coalesce，还是 freeze 掉 move 关系再进入 simplify/spill。前者看原始 move 边，后者只保留 family 外部仍然需要处理的 move 压力；
- **work_class / move_work_class / phase**: 这些字段决定候选进入哪个队列或阶段，例如 simplify、freeze、coalesce-ready、spill。也就是说，它们先决定“现在应该按哪类策略处理”，后面的 priority / move_work_priority / spill_cost 再在对应策略里排序；
- **preferred color**: select 阶段的颜色偏好。普通图染色只要找任意可用颜色即可；这里会优先尝试 partner/family 已经使用的颜色，以减少后续 move；

- **targeted eviction**: 当 preferred color 只被少数 blocker 挡住时, selector 会尝试定向挪开 blocker, 再保住当前 value 的 preferred color; 如果不安全, 才退回普通可用颜色或 spill/retry 路线;
- **spill_intended**、**spill_confirmed**、**spill_recovered**: 这些字段把 planner 的预测和 selector 的最终结果分开。它们让 rewrite 阶段只处理真正 confirmed spill, 也让 regression 能检查某个高风险值是否被 optimistic coloring 或 retry 成功救回。

priority 是这里最容易误读的字段。它不是一个全局绝对分数, 也不是“越大就一定越应该留在寄存器里”。当前实现大致把 affinity、interference degree、live block count 和 use count 等信号合成一个 base heuristic; 真正比较时还要看 phase:

- **simplify** 阶段优先选择已经低压、可安全摘除的 value。这里较高的 **priority** 或 **move-work priority** 更多表示“这个安全候选对 family/move 调度更有影响”, 而不是“它更危险”;
- **coalesce-ready** 阶段会把 partner weight、mutual-best、family size、external affinity 等信号加进 **move_work_priority**, 用来判断是否值得先合并 family, 避免太早 remove 导致错过共色机会;
- **freeze** 阶段处理的是“move 关系还在, 但暂时不继续 coalesce”的 family。freeze 会放弃这部分 move 优化偏好, 但 interference 约束仍然保留; 之后该 family/value 更容易回到普通 simplify 路线;
- **spill** 阶段不只看 **priority**, 还会优先比较 **spill_cost**、rematerializable、split-child、family pressure 等信号。**spill-remove** 也只是 planner 的高风险预测, 不等于最终 confirmed spill。

因此, **priority=40** 的 **simplify-remove** value 和 **priority=30** 的 **spill-remove** value 不能直接跨类比较。前者是在“安全摘除”队列里更值得安排, 后者是在“可能 spill”队列里风险较高; 二者表达的是不同阶段的调度含义。

下面先看一个 optimistic coloring 的例子。这个例子不是从某段 C 源码自然编译出来的, 而是 allocator 层的 regression witness。测试直接构造一个很小的 interference / coalescing 配置, 用来锁定“高风险值在 select 阶段如何通过 eviction/recolor 保住颜色”的策略。这样做的好处是变量少、压力关系清楚; 缺点是它不像源码例子那样直观, 所以需要先说明设定:

```
color budget:
  3 colors

coalesced families:
  family A: {ssa.1, ssa.3}, root = ssa.1
  family B: {ssa.2, ssa.5}, root = ssa.2

selected interference:
  ssa.0 conflicts with ssa.1, ssa.2, ssa.6
  ssa.1 conflicts with ssa.4
  ssa.3 conflicts with ssa.4, ssa.5
```

```

ssa.4 conflicts with ssa.5, ssa.6
ssa.5 conflicts with ssa.6

removal plan:
ssa.0: spill-remove, priority = 30
ssa.3: simplify-remove, priority = 8
ssa.1: simplify-remove, priority = 10
ssa.6: simplify-remove, priority = 40
ssa.5: simplify-remove, priority = 6
ssa.2: simplify-remove, priority = 10
ssa.4: simplify-remove, priority = 5

```

也就是说，这个 witness 故意制造了一个局面：ssa.0 是一个 spill-remove 高风险值；周围已经有多个值和 family 被染色；如果只做朴素逆序染色，它可能直接落入 spill 或破坏已有颜色安排。真实 dump 的关键片段如下：

读这个例子前先区分几个区域。coalesce 是 family 构造结果。removal plan 是 planner 的摘除顺序和 remove kind。select-trace 是 reverse-select coloring 的逐值记录。alloc func 是最终结果表。几个字段可以这样读：

- colors=3: 本轮可用颜色数，可以近似理解成 allocator 当前可分配的寄存器颜色预算；
- candidates=2: coalescing analysis 发现了两个可考虑合并/共色的 pair；
- weight: 该 coalesce pair 的 affinity 权重，越高表示越值得尝试保留共色偏好；
- can_coalesce=yes: 当前 interference 关系允许这对值成为 coalescing 候选；
- class ... root=... size=...: family 代表值和 family 大小，例如 ssa.2 代表 {ssa.2, ssa.5}；
- remove=simplify-remove: 该值在 planner 中按低压力 simplify 候选摘除；
- remove=spill-remove: 该值在 planner 中按高风险 spill 候选摘除，但不等于最终一定 spill；
- outcome=colored: select 阶段普通染色成功；
- outcome=optimistic-colored: 该值原本是 spill-remove 候选，但 select 阶段通过 optimistic 路线仍然找到了颜色；
- color=N / slot=N: 最终落到某个颜色，或者真的进入某个 spill slot；
- main-evict: 主 select 阶段为了当前值的颜色安排，选择某个 blocker 并尝试改色；本例中是把 ssa.2 挪到 color1。

在这个 witness 里，ssa.0 priority=30 spill-intended 的意思是：planner 曾经把 ssa.0 当作高风险 spill 候选摘掉，并记录了它的 spill priority；但最后 outcome=optimistic-colored 又说明 selector 成功把它救回了颜色。因此这里真正值得看的不是单独的分值，而是 remove、priority、outcome、main-evict 这几个字段合在一起描述的一次决策链。

```

coalesce manual_retry_blocker_family colors=3 candidates=2
  0: ssa.1 <-> ssa.3 weight=1 can_coalesce=yes
  1: ssa.2 <-> ssa.5 weight=3 can_coalesce=yes
  class ssa.1 -> root=ssa.1 size=2
  class ssa.2 -> root=ssa.2 size=2

select-trace colors=3 values=7
  0: ssa.4 remove=simplify-remove outcome=colored color=0
  1: ssa.2 remove=simplify-remove outcome=colored color=1
  2: ssa.5 remove=simplify-remove outcome=colored color=1
  3: ssa.6 remove=simplify-remove outcome=colored color=2
  4: ssa.1 remove=simplify-remove outcome=colored color=1
  5: ssa.3 remove=simplify-remove outcome=colored color=2
  6: ssa.0 remove=spill-remove outcome=optimistic-colored
    color=0 main-evict=ssa.2->color1

alloc func manual_retry_blocker_family colors=3 values=7
ssa.0 color=0 priority=30 spill-intended
ssa.1 color=1 priority=10
ssa.2 color=1 priority=10
ssa.3 color=2 priority=8
ssa.4 color=0 priority=5
ssa.5 color=1 priority=6
ssa.6 color=2 priority=40

```

这段 dump 的读法是：前半部分先说明 allocator 看见了两个 coalesced family；后半部分按 reverse-select 顺序打印每个 value 的最终结果。最后一行最关键：ssa.0 虽然是 spill-remove 摘下来的高风险值，但最后不是 confirmed spill，而是走到了 optimistic-colored。对应动作可以单独读成：

```

ssa.0 outcome=optimistic-colored
    main-evict=ssa.2->color1

```

也就是 selector 为了给 ssa.0 腾出可用安排，对 blocker ssa.2 执行了 recolor/evict 路线。

这里有一个细节需要讲清楚：select trace 中前面几行打印的是最终 allocation result，不是“被 eviction 修改前”的历史快照。所以 ssa.2 那一行已经显示为最终 color=1；真正说明发生过 recolor 动作的是后面的 eviction metadata。这里可以推导出 ssa.2 原先挡住的是 color=0：因为 ssa.0 最后 claim 到的颜色是 color=0，而 metadata 记录 blocker ssa.2 被挪到了 color=1。这个 metadata 不是 dump 字符串里的猜测，而是 allocation result 里的结构化字段：

```

used_generic_eviction = yes
blocker_recolored = yes
blocker_new_color = 1

```

对应 regression 也会通过 query helper 断言：对 ssa.0 使用了 eviction，blocker 是 ssa.2，并且 blocker 被 recolor 到 color=1。所以这个例子更适合放在 optimistic-colored 的位置：它说

明 `spill-remove` 只是 planner 的风险预测, 最终 `select` 仍然可以通过局部 `recolor` 把值救回寄存器。

真正体现 family-aware 的地方, 可以看下面这个 witness。这个 witness 在测试中名为 `manual_main_family`, 它把颜色数压到 2, 并且让目标值 `ssa.0` 同时面对两个可牺牲的 blocker:

```
color budget:
  2 colors

manual family facts:
  family A: {ssa.1, ssa.3}, root = ssa.1, size = 2
  family B: {ssa.2}, root = ssa.2, size = 1

selected interference:
  ssa.0 conflicts with ssa.1 and ssa.2
  ssa.1 conflicts with ssa.2

removal plan:
  ssa.0: spill-remove, priority = 30, spill_cost = 3
  ssa.1: simplify-remove, priority = 10, spill_cost = 2
  ssa.2: simplify-remove, priority = 10, spill_cost = 2
  ssa.3: simplify-remove, priority = 5, spill_cost = 2
```

这里关键不是 `ssa.0` 最后能不能 `optimistic-colored`, 而是为什么 selector 愿意为了它挪开 blocker, 以及当 `ssa.1` 和 `ssa.2` 都可以作为 blocker 被牺牲时 allocator 怎样选择。先看第一层原因: `ssa.0` 的 `spill_cost=3`, 而两个 blocker 的 `spill_cost` 都是 2, 所以牺牲一个较低 cost 的 blocker, 可以保住一个更贵的 spill 候选; 这就是 `eviction/recolor` 值得发生的原因。在纯标量 tie-break 里, 两个 blocker 的局部分数很接近, 甚至很容易退化成按 value id 或偶然顺序选择; family-aware 版本会多看一步: `ssa.1` 属于 `{ssa.1, ssa.3}` 这个 `size=2` 的 coalesced family, 而 `ssa.2` 只是一个 singleton family。因此主 select 的 generic eviction 会优先牺牲更小的 family, 也就是 `ssa.2`:

```
select-trace colors=2 values=4
  1: ssa.2 remove=simplify-remove
    spill_cost=2(uses=1,blocks=1,affinity=0)
    outcome=spill-confirmed
  3: ssa.0 remove=spill-remove
    spill_cost=3(uses=2,blocks=1,affinity=0)
    outcome=optimistic-colored color=0

allocation checks:
  ssa.0 -> color0
  ssa.1 -> colored, not spilled
  ssa.2 -> spill-confirmed
```

这个例子说明 family-aware 不是把 family 合成大节点, 而是在牺牲 blocker 前, 先看是否值得用低代价 blocker 保住高代价 value, 再参考 `recolor` 可行性、family 当前颜色支持和 `pressure`。

前一个例子偏 optimistic-colored，这个例子才体现 family-aware 的 blocker 取舍。

和传统“放进栈里、逆序染色”的朴素版本相比，这里的差别不在于栈式 select 消失了，而在于 select 时多了可解释的偏好、恢复动作和 family 取舍。朴素实现通常只回答“这个点有没有颜色”；这个 allocator 还能回答“为什么它想要这个颜色、是谁挡住了它、有没有把 blocker 挪开、牺牲哪个 blocker 对 family 破坏更小、最后是不是真的 spill”。在热点代码里，这类策略可以减少无意义 move，避免把本来属于同一 value family 的值随意拆散；更重要的是，它让 allocator 的决策能被 dump 和 regression test 直接锁住，而不是依赖主观判断。

这种设计带来的工程收益主要有：

- move-related value 不再只是零散边，而是可以作为一组对象被观察；
- coalescing、freeze、spill 的原因可以通过 trace/report 解释；
- spill 后可以进行真实 rewrite，而不是只在报告里标记失败；
- no-move 函数可以走 fast path，避免为简单函数硬跑完整 family machinery；
- rewrite loop 可以只重分配被改写过的函数，降低不必要的编译时间。

4.4 效果与边界

-perf 模式在最终性能评测中通过了全部 20 个性能用例，正确性分数为 100.00。榜单记录为第 2 名，总用时 103.72s。单项结果如下：

用例	结果	用时 / s
00_bitset1	AC	2.110806
01_bitset2	AC	4.321101
02_bitset3	AC	6.548924
03_mm1	AC	7.327778
04_mm2	AC	6.220852
05_mm3	AC	4.664204
06_mv1	AC	6.017828
07_mv2	AC	3.883102
08_mv3	AC	3.756907
09_spmv1	AC	2.000937
10_spmv2	AC	1.282645
11_spmv3	AC	1.530294
12_fft0	AC	2.949113
13_fft1	AC	6.610910
14_fft2	AC	6.063861
15_transpose0	AC	2.295379
16_transpose1	AC	2.728154
17_transpose2	AC	5.738144
18_brainfuck-bootstrap	AC	11.708512
19_brainfuck-calculator	AC	15.961799

表 1: -perf 模式最终性能评测结果

-perf 模式已经形成了比较完整的优化实验框架，但它并不宣称拥有一个完全成熟的工业级优化器。当前更准确的定位是：

- 已有不少真实、有回归保护的中端优化；
- 一些优化是 general pass，一些仍是 hotspot-specific pass；
- 部分 loop/induction 优化对具体 witness 有收益，但需要 A/B 判断是否保留；
- 编译时间同样是评价指标，不能为了小幅 runtime win 引入过高 compile-time cost。

5 -extension 模式：语言扩展与 callable-object IR

-extension 是项目中最有语言设计味道的一条线。它既是功能开关，也是保守翻译管线开关。很多扩展特性只在该模式下开放，避免影响 -riscv 和 -perf 的课程基础目标。

5.1 扩展特性概览

当前 -extension 线覆盖了多类语言扩展：

- control-flow extensions: defer、fndefefer、capdefer；
- function-value family: non-capturing function values、local alias/reassignment、branch re-binding、function-valued parameters、returned callable、higher-order forwarding；
- closure/callable family: closure literal、local closure、returned closure、closure captures callable，以及后续的 callable-object IR；
- data/number extensions: pair、struct、aggregate 参数与返回，以及 conservative float slice。

这些特性不是一次性“全开”，而是按 source family 分阶段落地。每打开一个新形状，就补 parser/semantic/IR/lower/runtime 等回归测试。项目里很多看似特殊的小限制，本质上都是为了让每个 feature slice 都能稳定闭环。

5.2 defer / fndefefer / capdefer

defer 这一组扩展主要解决“退出某个作用域时再执行一段代码”的问题。普通 defer 是 scope-exit 语义：注册点之后，当控制流离开当前作用域时，延迟语句按栈式顺序执行。它和 Go 里的直觉类似，但项目里特意保守处理了控制流边界：defer body 中不能直接 return、break 或 continue 跳出外层；如果 return expr 遇到 active defer，会先固定返回值，再运行 deferred side effects。这样可以避免“返回值到底是注册时、退出时还是副作用后”这类语义不确定性。

一个最小例子如下：

```
int main() {
    int x = 1;
    defer putint(x);
    x = 2;
```

```

defer putint(3);
return 0;
}

```

这里输出顺序是 3, 2: 后注册的 `putint(3)` 先执行; 前面的 `putint(x)` 读取的是退出时的 `x`, 所以输出 2。

`fndeferr` 是函数退出级别的 `defer`。它不绑定某个小 block 的退出, 而是等函数即将返回时统一运行; 普通 block-level `defer` 会先按作用域 unwind, 随后再运行函数级 `fndeferr`。这个特性适合表达“无论从函数哪里返回, 都要在最后做一件事”的模式, 例如:

```

int g;
int main(int x) {
    int y = 0;
    fndeferr { putint(g); putint(x); putint(y); }
    y = 3;
    return 0;
}

```

如果调用 `main(5)` 且全局 `g` 为 0, 那么函数返回前会输出 0, 5, 3。这说明当前 `fndeferr` 读的是函数退出时仍然稳定可见的名字, 而不是注册瞬间的快照。

当前 `fndeferr` 不是任意闭包捕获系统, 而是一个保守的函数退出注册机制。它要求引用的名字在函数级视角下稳定可见; 嵌套 block local、loop local、尚未声明的 later local 都会被拒绝。`fndeferr` body 中也不允许再嵌套 `defer/fndeferr` 或制造新的 `return/break/continue` 控制转移。和循环有关的限制更微妙: 项目后来支持了部分动态注册形状, 例如在 `if` 中按路径注册一次 `fndeferr`, 也支持一个 loop-nested 注册点被重复执行; 实现上会用隐藏计数器记录该 site 执行了多少次, 并在函数退出时 replay 对应次数。但当前仍只支持每个函数一个这样的 loop-nested site, 不是完整的运行时 defer stack。

例如, 下面这种条件注册是可以表达的:

```

int main() {
    int x = 1;
    if (x) { fndeferr putint(x); }
    x = 4;
    return 0;
}

```

输出是 4。但下面这种引用 loop-local 的写法会被拒绝:

```

int main() {
    for (int i = 0; i < 3; i = i + 1) {
        fndeferr putint(i); // rejected: loop-local name
    }
    return 0;
}

```

还有一种拒绝和名字作用域无关, 而是来自动态注册机制本身: 同一个函数里出现多个 loop-nested `fndeferr` site 时, 当前实现会拒绝, 即使它们只引用全局变量:

```
int a[8];
int main() {
    for (int i = 0; i < 8; i = i + 1) {
        fndefefer putint(a[2]);
        fndefefer putint(a[3]); // rejected: second loop site
    }
    return 0;
}
```

对应的设计取舍是：单个重复注册点可以被编译成“计数器 + 退出时循环 replay”，而多个动态注册点需要更接近 runtime stack 的顺序记录。这里先 gate 住，避免半支持导致退出顺序不可靠。

capdefer 则是对这个缺口的一个小而明确的补充：它显式把表达式按值捕获到新名字里，再在退出时执行延迟语句。例如：

```
int main() {
    int x = 1;
    capdefer(y = x) putint(y);
    x = 2;
    return 0;
}
```

这里延迟执行时打印的是注册时捕获的 `y`，而不是退出时已经变成 2 的 `x`。因此，`defer` 偏 exit-time state，`capdefer` 偏 registration-time snapshot。上面例子的输出是 1；如果换成普通 `defer putint(x)`，输出就会是 2。`capdefer` 同样保持保守边界：它只在 `-extension` 下开放，循环注册和引用嵌套 block-local 的危险形状会被拒绝。例如：

```
int main() {
    int x = 1;
    for (; x < 2; x = x + 1) {
        capdefer(y = x) putint(y); // rejected: dynamic capdefer registration
    }
    return 0;
}
```

5.3 pair、struct 与 aggregate

`pair/struct` 这一组扩展表面上是在给语言增加复合值，工程上更重要的点则是：项目在不引入完整对象 ABI 的前提下，先让复合值以保守、可验证的方式流动起来。

`pair` 是最小的 builtin aggregate，支持局部声明、{1, 2} 初始化、copy-init/assignment，以及 `.first/.second` 字段访问。`struct` 则把这条线推进到 named aggregate：用户可以在顶层声明类型，再在局部创建对象、初始化、赋值和访问字段。例如：

```

struct Point { int x; int y; int z; };

int main() {
    pair p = {1, 2};
    struct Point q = {3, 4, 5};
    q.x = p.first + q.y;
    return q.x + p.second;
}

```

这里的实现并不是把 `p` 或 `q` 变成一个运行时对象，而是分层识别、再递归展平成 hidden scalar slots。具体可以拆成四步。

第一步，parser/AST 保留两类信息：顶层 `struct` 定义会进入 `AstProgram` 的 struct type table；字段访问则是普通的 member expression，形如 `base.field`。所以 `b.p.second` 在 AST 中不是一个扁平字符串，而是一条 member chain。

第二步，semantic 根据 scope 中记录的 declaration kind 和 type name 识别 aggregate root。`pair` 的合法字段固定是 `first/second`；`struct` 则按 struct type table 查字段名。如果字段本身还是 `pair` 或另一个 known `struct`，semantic 返回的仍然是 aggregate type descriptor，于是下一层 `.second` 或 `.x` 还能继续递归解析。

第三步，IR lowering 给每个 aggregate local 分配一段连续 scalar locals，并在 lowering scope 里记录：

```

aggregate binding =
    first_local_id + aggregate_slot_count + aggregate_kind + type_name

```

因此 lowering 心智模型更接近：

```

pair p = {1, 2}
-> p.first_slot = 1
-> p.second_slot = 2

struct Point q = {3, 4, 5}
-> q.x_slot = 3
-> q.y_slot = 4
-> q.z_slot = 5

```

第四步，字段访问才真正“用起来”：lowering 先解析 member chain 的 base 是哪个 aggregate binding，再根据 struct 字段顺序计算 flattened offset。对 `pair` 来说，`first` 是 offset 0，`second` 是 offset 1；对 `struct` 来说，某个字段的 offset 等于它前面所有字段 flatten 后的 slot 数之和。

这个策略自然支持嵌套 aggregate，例如：

```

struct Box { pair p; int tag; };

int main() {
    struct Box b = {{1, 2}, 7};
    pair q = b.p;
    return q.second + b.tag;
}

```

```
}

```

在这个例子里，Box 的 flatten layout 可以理解为：

```
b.p.first  -> slot 0
b.p.second -> slot 1
b.tag      -> slot 2

```

所以 `b.p.second` 的解析过程是：先确认 `b` 是 `struct Box`，再确认字段 `p` 是 `pair`，于是 `b.p` 变成一个 aggregate-valued sub-binding；最后 `second` 映射到这个 sub-binding 的第 1 个 scalar leaf。copy-init 和 assignment 也是逐 leaf 搬运，而不是复制一个 opaque object。真实 IR witness 里也能看到这种形状，例如 `struct Mid { pair p; int z; }` 会展开出类似 `m$p$0`、`m$p$1`、`m$z` 的 locals。

这条线后来还保守地打开了函数边界：`pair` 和 `known struct Name` 可以作为参数和返回值出现在若干受控 family 中，例如：

```
pair id(pair p) { return p; }
int sum(pair p) { return p.first + p.second; }
pair mk() {
    pair p = {10, 20};
    return p;
}
int main() {
    pair q = id(mk());
    return sum(q);
}

```

这仍然不是完整 C aggregate ABI。更准确地说，caller/callee 共享同一套 flatten contract：aggregate 参数会展开成 hidden scalar parameter sequence，aggregate 返回则通过 hidden result slots 写回。也就是说，跨函数边界流动的是 leaf slots，不是一个任意位置都可作为普通表达式消费的 first-class aggregate object。

因此当前边界也很明确：aggregate globals、aggregate arrays、任意 scalar context 中的 plain aggregate value，以及直接对调用结果取字段的形状仍然会拒绝。比如 `mk().second` 当前不会被当作“临时 aggregate 对象上读字段”自动打开；更推荐、也更符合当前 lowering contract 的写法是：

```
pair q = mk();
return q.second;

```

这正是本项目处理语言扩展的一贯风格：先把 type descriptor、field lookup、flatten order、copy/return/param compatibility 这些基础设施收稳，再决定是否进入真正的一等 aggregate ABI。这种小步推进可以让扩展能力保持可验证，而不是在 ABI 尚未稳定时扩大语义表面。

5.4 函数值与闭包

函数值主线是 -extension 最值得重点介绍、也最容易误解的部分。它的目标不是一次性实现完整 C 函数指针或 ML 风格闭包运行时，而是在当前编译器架构里逐步回答三个问题：

1. 函数是否可以作为值传给另一个函数；
2. callable 离开定义点之后，在返回、赋值、转发、捕获时是否还能被正确恢复；
3. 这些恢复动作是否能从 lowering context 里的隐式约定，收敛成 IR/verifier 可见的对象合约。

因此实现过程不是先引入完全通用的 function object，而是分阶段推进。

第一阶段：non-capturing specialization。 最早支持的是无捕获、目标静态已知的函数值参数：

```
int add1(int x) { return x + 1; }
int apply(int f(int), int x) { return f(x); }
int main() { return apply(add1, 41); }
```

这一层的核心不是“已经有通用函数指针 ABI”，而是 specialization：调用点知道 `f=add1`，于是构造一个 specialized helper。心智模型大致是：

```
apply(add1, 41)
  -> apply__fv_0_add1(41)
  -> call add1(41)
```

后续的 local alias、forwarding、多 function-valued parameters、zero-argument、void return 等 sibling 都是在这条线上扩张。比如：

```
int compose(int f(int), int g(int), int x) {
    return f(g(x));
}
```

如果 `f` 和 `g` 在调用点都是已知 top-level 函数，lowering 就可以生成对应组合的 specialized helper。

第二阶段：dynamic noncapturing 与 returned callable。 一旦出现运行时选择，单纯静态 specialization 就不够了：

```
int add2(int x) { return x + 2; }
int main(int c) {
    int f(int) = add1;
    if (c) f = add2;
    return f(40);
}
```

这类值当前用 target family/tag 表达。编译期先为这个 local 推出一个有限 target set，例如 {add1, add2}，并给每个 target 一个内部 tag；local 本身会多出一个 hidden tag slot，dump 里常见名字类似 `f$ftag`。初始化和赋值并不复制一个真实函数对象，而是更新这个 tag：

```
int f(int) = add1;  -> f$ftag = tag(add1)
f = add2;           -> f$ftag = tag(add2)
```

调用 `f(40)` 时，lowering 读取 `f$ftag`，按 `tag` 走不同分支。每个分支仍然落到编译期已经生成好的 specialized leaf:

```
if f$ftag == tag(add1):
    call __fnwrap_add1(..., 40)
else if f$ftag == tag(add2):
    call __fnwrap_add2(..., 40)
```

如果这个动态 local 被传给 `apply(f, 40)`，也是同一个思路：先看 `f$ftag`，再选择某个已生成的 `apply` specialized helper:

```
tag(add1) -> apply__fv_0_add1(40)
tag(add2) -> apply__fv_0_add2(40)
```

所以第二阶段仍然不是 generic runtime function pointer；它是“有限 target family + hidden tag + branch-selected specialized helper”。

再往前一步，函数值还可以跨 return boundary:

```
int pick(int c)(int) {
    if (c) return add2;
    return add1;
}
int main() {
    int f(int) = pick(1);
    return f(40);
}
```

这里真正的工作不是 `parse int pick()(int)`，而是 producer 返回一份 conservative callable payload，caller 再 materialize 本地 callable view。相关修复集中在这里：同一个 returned producer 不能重复求值，ternary/passthrough/wrapper 里要找回真正 producer，多个 function-valued actual argument 还要组合它们的 dispatch。

第三阶段：closure 与 capture payload。 closure 让问题从“返回哪个函数”升级为“返回哪个函数以及哪些捕获值”。第二阶段里 `f$ftag` 基本只需要回答“现在是哪一个 non-capturing target”；第三阶段则要同时回答：“现在是哪一个 closure helper”，以及“这个 helper 运行时要看见哪一组 captured values”。本地 closure 的基本形状是：

```
int main() {
    int x = 3;
    int f(int) = closure [x] int (int y) { return x + y; };
    return f(4);
}
```

lowering 会先把 closure literal 拆成一个 helper/wrapper，再把 capture list 拆成 hidden payload。对上面的例子，可以把 local `f` 想成被展开成了两类隐藏状态：

```
f$ftag          = tag(main__closure_f)
f$closurecap$0  = x
```


`f$ftag` 表示这个 local 当前绑定到哪一个 closure family; `f$closurecap$0` 保存第 0 个捕获值，也就是初始化时的 `x`。这仍然是 by-value capture：之后外层 `x` 如果再被赋值，不会自动改写已经保存进 payload 的值。

调用 `f(4)` 时，lowering 不会再去“按名字查外层的 `x`”。它先根据 `f$ftag` 找到 closure wrapper，再把 capture slot 当作 hidden env 传进去：

```
env = f$closurecap$0
obj = fn_make __fnwrap_closure_main__closure_f, env, shape(int)->int
call_indirect obj(4)
```

wrapper 的物理调用形式可以理解成 `code(env, visible_args...)`，所以 closure body 里的 `x` 是从 env/capture payload 中恢复出来的。这里的关键变化是：source 里只有 `f(4)` 一个参数，但 wrapper 实际上会多吃一个 hidden env；因此后续 verifier 和 lower 阶段必须一直区分 hidden 参数和 source-visible 参数。

returned closure 进一步要求 payload 跨函数边界：

```
int make(int x)(int) {
    return closure [x] int (int y) { return x + y; };
}
int main() {
    int f(int) = make(3);
    return f(4);
}
```

这个例子里，`make(3)` 不能只返回一个 tag。它至少要把 closure family identity 和捕获值 `x=3` 一起带回 caller。当前实现仍然避免完整 heap closure runtime，而是使用保守的 hidden return payload。可以将其概括为：

```
callee make:
    __retclosure_tag      = tag(make__retclosure...)
    __retclosure_cap_0    = x

caller main:
    f$ftag                = __retclosure_tag
    f$closurecap$0        = __retclosure_cap_0
```

真实 dump 中的名字会更工程化，常见前缀包括：

```
__retclosure_*
__retclosure_argcap_*
__retclosure_immediate_*
__retclosure_specarg_*
```

这些名字的职责是把 returned producer 产生的 family tag 和 capture slots 稳定地交给 caller。之后 `f(4)` 就和本地 closure 调用走同一条 wrapper / callable-object 路径。

当前还支持一些更复杂但仍保守的 family，例如 closure 捕获 callable：

```
int wrap(int f(int), int x) {
```

```
int g(int) = closure [f] int (int y) { return f(y); };
return g(x);
}
```

这时 capture payload 里不只有 scalar，还可能有 function-value tag / callable payload。也就是说，closure [f] 捕获的不是一个普通整数值，而是“当前 f 代表哪个 callable family”的内部 payload；closure helper 进入 body 后还要恢复 f(y) 这个调用。实现上因此需要：

```
capture time:
  g$closurecap$0 = f$ftag or rebuilt callable payload

inside closure helper:
  recover captured callable
  dispatch captured f(y)
```

这也是为什么这条线后来不再适合只依赖 helper 名称拼接：payload 可能来自 local binding、returned producer、passthrough wrapper 或多个 function-valued actual argument，必须有统一的 materialize/rebuild 规则，否则容易出现同一个 producer 被重复求值、不同 payload prefix 指向不同临时槽等真实错误。

第四阶段：callable-object IR。 随着 returned callable、returned closure、closure captures callable、higher-order passthrough 越来越多，项目把共同抽象收敛成 canonical IR 中显式可见的 callable object。前三阶段仍然可以理解成在 source lowering 里维护 hidden tag、capture slot 和 helper；第四阶段要做的是把这些约定变成 IR 里能 dump、能 verify、能继续 lowering 的对象：

$$\text{callable object} = (\text{code}, \text{env}, \text{shape})$$

它对应三个字段：code 是 wrapper/helper 入口，env 是 hidden capture payload，shape 是 source-visible signature。non-capturing function、local closure 和 returned closure 都可以被映射到同一种构造动作：

```
static function add1:
  code = __fnwrap_add1
  env  = 0

local / returned closure:
  code = __fnwrap_closure_...
  env  = f$closurecap$0 or returned capture payload

object:
  obj = fn_make code, env, shape
```

在 canonical IR 中，最小调用形态是：

```
shape shape.0(int) -> int
tmp.0 = fn_make __fnwrap_add1, 0, shape.0
tmp.1 = call_indirect tmp.0(41)
```

这里 `shape` 专门描述 visible signature, 例如 `(int)->int`; 它不包含 hidden env。同时, indirect call 也只接收 visible args。这个区别很重要: closure 可能有 env, 但调用参数数量仍然只看 source-level 形参。因此 verifier 可以检查三件事:

- `fn_make` 指向的 wrapper/helper 是否存在;
- wrapper 的物理参数是否等于 `hidden env + visible args`;
- `call_indirect` 的 visible arg count 是否匹配 `shape`。

这也是 zero-arg closure、void closure 等 case 容易出 bug 的原因: env/capture slot 数量可以不为 0, 但 visible arg count 必须仍然是 0。

lower-IR/backend 不一定一直保留 `call_indirect` 文本形状。如果三元组已经明确, 可以桥接成:

```
call __fnwrap_closure...(env, visible_args...)
```

所以正文或 dump 里看到 direct wrapper call, 并不代表第四阶段没有发生; 它通常表示 canonical IR 先建立了显式 callable-object contract, 后续层再把它安全地折回普通 call。

这条 IR 线的意义在于: callable 不再只藏在 lowering context 和 payload slot 约定里, 而是变成 dump 可展示、verifier 可检查、lower/backend 可桥接的显式对象。后端如果证明 `code/env` 已知, 可以再把它折回直接 wrapper call; 这不是倒退, 而是“IR 合约显式化, 后层允许优化”的正常分工。

higher-order: 有限层显式签名, 而不是任意泛型。项目还支持了一批 higher-order 形状, 例如函数参数本身也接收函数参数:

```
int apply(int f(int), int x){ return f(x); }
int pass(int h(int f(int), int x), int f(int), int x){
    return h(f, x);
}
```

这一类依赖 parser/semantic 保留 nested function signature metadata; semantic 递归比较 return kind、parameter count 和 nested function parameter shape; lowering 再用 passthrough 分析和 recursive dispatch, 把 outer callable binding 传到 leaf callable-object dispatch。所以它不是自动泛型 lambda, 而是“有限层显式 shape + specialization/callable-object collapse”。

当前能写什么。 概括地说, 当前 -extension 已能覆盖这些代表性 family:

- top-level/builtin function 作为 function-valued parameter, 含多参数、zero-arg、void sibling;
- local function-typed alias、部分 reassignment、branch rebinding 和 dynamic tag dispatch;
- noncapturing function-valued return, 以及 bind-and-call / immediate-call 消费;
- local closure、returned closure、multi-capture closure, 以及部分 closure reassignment / alias;

- closure literal 或 returned closure 作为 function-valued actual argument;
- closure 捕获静态或参数传入的 function value;
- 显式 nested signature 的 second/third/... order passthrough family;
- canonical IR 上的 `shape + fn_make + call_indirect` callable-object 路径。

当前不能写什么。 同样重要的是边界。当前项目仍不声称支持：

- 完整 generic function pointer ABI 或所有 callable 都可任意存取的 runtime;
- callable value 存进 global、array、aggregate field 后再自然流动;
- 任意深度、任意表达式形态的 CPS/continuation 程序;
- polymorphic/auto lambda、自递归 callable、真正无限层 higher-order type inference;
- 所有 closure literal + function-valued-parameter 组合，尤其是需要完整动态 continuation stack 的形状;
- 完整 heap escaping closure lifetime 模型。

换句话说，当前实现拥有的是一条很宽的 conservative function-value mainline，而不是最终形态的 first-class function runtime。这个边界反而是设计的一部分：支持的 family 用 regression 锁住；暂时不支持的 family 尽量在 semantic gate 处明确拒绝，避免把未完成能力暴露成不稳定行为。

6 端到端翻译案例

为了避免报告只停留在结构描述上，本节选取几段小而有代表性的 `-extension` 程序，展示它们在不同阶段中的关键形状。这里不把所有扩展硬塞进同一个综合样例，因为某些组合本来就是当前 semantic gate 会保守拒绝的；更清楚的方式是：用一个主 showcase 展示 aggregate + closure + callable-object 主线，再用一个补充 showcase 展示 defer/capdefer/fndefefer 的控制流改写。正文只展示关键片段，以避免大段 dump 干扰主线叙述。

6.1 主 showcase: aggregate + returned closure

```
struct Box { pair p; int bias; };

int sum_box(struct Box b) {
    return b.p.first + b.p.second + b.bias;
}

struct Box mkbox(int a, int b, int c) {
    struct Box box = {{a, b}, c};
    return box;
}
```

```

int apply(int f(int), int x) { return f(x); }
int pass(int h(int f(int), int x), int f(int), int x) {
    return h(f, x);
}
int make(int base)(int) {
    return closure [base] int (int y) { return base + y; };
}
int main() {
    struct Box b = mkbox(1, 2, 3);
    int f(int) = make(sum_box(b));
    return pass(apply, f, 4);
}

```

这个例子刻意覆盖几类 `-extension` 形状，但仍保持可读。`Box` 里嵌了 `pair`；`mkbox` 通过 hidden return slots 返回 aggregate；`sum_box` 通过 flattened aggregate 参数读取字段。随后程序用 `sum_box` 的结果构造 returned closure，再通过 `pass/apply` 展示 higher-order passthrough 被折到 leaf callable dispatch。

6.2 aggregate lowering 关键片段

```

func sum_box(b.0, b.1, b.2) {
    bb.0:
        tmp.0 = mov b.0
        tmp.1 = mov b.1
        tmp.2 = add tmp.0, tmp.1
        tmp.3 = mov b.2
        tmp.4 = add tmp.2, tmp.3
        ret tmp.4
}

func mkbox(__ret0.0, __ret1.1, __ret2.2, a.3, b.4, c.5) {
    bb.0:
        box$p$0.6 = mov a.3
        box$p$1.7 = mov b.4
        box$b$bias.8 = mov c.5
        0 = store_indirect __ret0.0, box$p$0.6
        0 = store_indirect __ret1.1, box$p$1.7
        0 = store_indirect __ret2.2, box$b$bias.8
        ret
}

```

这段说明 aggregate 当前不是走完整 C ABI 的“结构体整体对象”，而是被拆成 leaf scalar slots。`struct Box { pair p; int bias; }` 被展开为 `p.first`、`p.second`、`bias` 三个槽；作为参数时表现为 `b.0/b.1/b.2`，作为返回值时则通过 `__ret0/__ret1/__ret2` 这类 hidden return

slots 写回 caller。

6.3 callable-object 关键片段

```
shape shape.0(int) -> int

func main() {
  bb.0:
    tmp.0 = addr_local local.0
    tmp.1 = addr_local local.1
    tmp.2 = addr_local local.2
    tmp.3 = call mkbox(tmp.0, tmp.1, tmp.2, 1, 2, 3)
    tmp.4 = addr_local local.3
    tmp.5 = call sum_box(b$p$0.0, b$p$1.1, b$bias.2)
    tmp.6 = call make(tmp.4, tmp.5)
    f$ftag.4 = mov 1
    __closure_env_0.5 = mov f$closurecap$0.3
    tmp.7 = addr_local local.5
    tmp.8 = fn_make __fnwrap_closure_make__retclosure_7_33_1,
                  tmp.7, shape.0
    tmp.9 = call_indirect tmp.8(4)
    ret tmp.9
}
```

这段把几条线连在一起了:mkbox 先把 aggregate 写到 caller 准备的 hidden slots;sum_box(b) 再按 flattened slots 读出字段;make(...) 把求出的 base 保存为 returned closure payload;最后 fn_make 与 call_indirect 把 closure wrapper、环境地址和 callable shape 合成 verifier 可见的 callable object。这里源码写的是 pass(apply, f, 4), 但 passthrough 分析已经把外层 higher-order shell 折到 leaf callable dispatch, 所以 IR 里直接出现最终 closure 调用路径。

6.4 lower IR 与 RISC-V 关键片段

```
func main() {
  bb.0:
    tmp.0 = addr_local b$p$0.0
    tmp.1 = addr_local b$p$1.1
    tmp.2 = addr_local b$bias.2
    call mkbox(tmp.0, tmp.1, tmp.2, 1, 2, 3)
    tmp.11 = load_local b$p$0.0
    tmp.12 = load_local b$p$1.1
    tmp.13 = load_local b$bias.2
    tmp.5 = call sum_box(tmp.11, tmp.12, tmp.13)
    call make(addr_local f$closurecap$0.3, tmp.5)
    store_local f$ftag.4, 1
    tmp.9 = call __fnwrap_closure_make__retclosure_7_33_1(
```

```

        addr_local __closure_env_0.5, 4)
    ret tmp.9
}

```

lower IR 的第一件事，是把 local/global 访问显式拆成 `load_*` 与 `store_*`。与此同时，当前 slice 会把 `call_indirect` 桥接为 wrapper call。最终输出的 RISC-V 片段也能看到同样的结构：

```

main:
    mv a0, sp
    addi a2, sp, 4
    addi a1, sp, 8
    call mkbox
    call sum_box
    mv a1, a0
    addi a0, sp, 12
    call make
    ...
    li a1, 4
    call __fnwrap_closure_make__retclosure_7_33_1
    ret

```

这个案例体现了项目的一个核心风格：源码、IR、lower IR 和汇编之间不是黑箱跳跃，而是每一层都有能解释的中间形状。对复杂语言特性来说，这比“最后能跑”更重要；最终运行结果仍然必要，但中间层契约也必须能够解释。

6.5 defer / capdefer / fndefer 展示

另一类扩展不主要改变值的表示，而是改变 `return` 前的控制流。一个小例子是：

```

int main() {
    int x = 1;
    defer { x = x + 10; }
    capdefer(y = x) putint(y);
    x = 2;
    fndefer putint(x);
    return x;
}

```

对应 canonical IR 的关键形状如下：

```

func main() {
    bb.0:
        __fndefer_count_0.0 = mov 0
        x.1 = mov 1
        __capdefer_0_y.2 = mov x.1
        x.1 = mov 2
        __fndefer_count_0.0 = mov __fndefer_count_0.0 + 1
        __fndefer_ret.3 = mov x.1
}

```

```

    x.1 = mov x.1 + 10
    call putint(__capdefer_0.y.2)
    __fndefefer_i.4 = mov 0
    jmp replay_fndefefer
replay_fndefefer:
    if __fndefefer_i.4 < __fndefefer_count_0.0:
        call putint(x.1)
        __fndefefer_i.4 = __fndefefer_i.4 + 1
        jmp replay_fndefefer
    ret __fndefefer_ret.3
}

```

这段展示了三者的差别：`capdefer` 在注册点保存 snapshot，所以打印的是旧的 `x=1`；普通 `defer` 在 `return` 前展开并修改当前 `x`；`fndefefer` 则通过 hidden counter 和 replay loop 表示函数退出级注册。注意返回值先被保存到 `__fndefefer_ret`，所以 `defer` 修改 `x` 不会意外改变本次 `return` `x` 已经确定的返回值。

7 严格返回路径审计

除了三个主模式外，项目还提供了一个可叠加的 strict gate:

```
--enforce-all-paths-return-check
```

它不是第四种编译模式，而是一个更严格的语义审计开关。默认 driver 会跳过这个检查，是为了兼容课程测试中较宽松的非 `void` 函数写法；但在需要审计控制流时，可以显式打开它，让 semantic 拒绝没有 all-path return 证明的函数。

7.1 为什么需要单独做这个检查

只检查“函数里出现过 `return`”是不够的。下面这个函数有 `return`，但显然不是所有路径都返回：

```

int f(int x) {
    if (x) return 1;
}

```

当 `x==0` 时，程序会直接落到函数末尾。对非 `void` 函数来说，这就是未定义的返回值来源。课程基础模式为了兼容性默认放宽；strict gate 则把它作为明确的语义错误报告为 SEMA-CF-001。

7.2 实现思路

这个检查不是运行时检查，也不是完整停机证明，而是在语义分析阶段对 statement AST 做控制流摘要。实现上会计算两个核心信息：

```
returns_all_paths = guaranteed_return && !may_fallthrough
```


`guaranteed_return` 表示当前语句树是否形成必然返回边界；`may_fallthrough` 表示是否仍有路径能走到函数末尾而不返回。只有非 `void` 函数满足上式，`strict` 模式才接受。这个摘要会递归处理 `compound`、`if/else`、`while`、`for`、`break/continue` 等控制流；它的目标不是证明所有程序性质，而是保守、稳定、可解释。

7.3 循环与不可达 `return`

循环形状能较清楚体现这个检查的价值。例如：

```
int f(int a) {
    while (1) {
        if (a) return 1;
    }
}
```

它后面即使补一个不可达的 `return`，也不能伪装成所有路径都返回；因为 `a==0` 时会进入一个静态可见的非返回路径。相对地，如果循环体内部已经覆盖所有分支：

```
int f(int a) {
    while (1) {
        if (a) return 1;
        else return 2;
    }
}
```

则 `strict gate` 会接受。类似地，`for(;;)` 中如果所有可达分支都会返回，也可以通过；如果只有部分分支返回，则会被拒绝。这一点避免了用无限循环后方的不可达 `return` 掩盖真实的非返回路径。

7.4 边界与测试价值

这个检查拒绝的不是所有潜在风险程序，而是在当前保守分析下不能证明 `all-path return` 的程序。它也不会声称解决停机问题：值依赖、调用依赖、复杂循环退出等情况会保持谨慎，避免把合法程序误杀。项目的测试中也专门区分了稳定 `reject`、保守 `accept` 和 `deterministic loop reject` 几类 `case`，防止后续语义分析改动把这个 `gate` 变得过宽或过窄。

从工程角度看，`all-paths-return` 的意义不只是多报一个错。它给前端控制流分析提供了一个小而清楚的证明任务：哪些路径会返回，哪些路径可能 `fall through`，哪些循环不能被不可达的后续 `return` 误判为已覆盖。编译器可以允许默认模式保持兼容性，但 `strict` 模式必须提供更严格的语义审计。

8 Vibe 工作流与质量保障

本项目开发过程中采用了比较典型的快速迭代式 `vibe coding` 工作流：先围绕一个具体 `witness` 或 `feature slice` 形成假设，再实现最小改动，随后立即通过测试和实际运行验证。这种方式的好处是推进快，风险是容易在复杂 `lowering` 中产生仅由最终现象支撑的错误判断。因此项目把质量保障放在非常靠前的位置。

8.1 AI agent 协作设定

本项目的特殊之处，是它几乎完全通过 AI agent 协作完成代码层面的推进。人的角色并不是逐行写 C 代码，而是负责路线、目标、优先级和大体实现方法：决定当前要攻哪一簇问题、什么算真正修好、哪些风险需要暂停，以及什么时候必须跑全量测试。在实际开发中，项目负责人参与了架构决策、验收和方向控制，但基本不直接书写实现代码；几十万行级别的实现主要由 agent 根据目标拆解、阅读代码并落地。

协作大致分成三类 agent：

- 实现 agent：阅读当前权威文档和代码，负责在 `src/`、`include/`、`tests/`、`docs/` 内实现功能、修 bug、补回归测试；
- review/lesson agent：阅读代码和测试结果，做代码 review、归纳项目结构，并将复杂实现压缩成 `lesson/` 中更容易复用的说明；
- planning/hacking agent：阅读代码、lesson 和路线文档，精炼下一步计划，同时主动寻找当前实现或规划中可能有问题的地方。

这些 agent 不是随意开工。仓库中的 `AGENTS.md` 定义了启动顺序、默认角色、作用域边界、测试纪律和版本规则。例如 agent 开始工作前需要读取 `AGENTS.md`、`docs/ENGINEERING_MEMORY.md` 和 `docs/NEXT_STEPS.md`；如果进入优化线，还要读取优化计划和 reusable pass 待办。它还明确规定了不能随意修改 `lesson/`、不能提交已知 broken code、性能改动必须做 A/B、长命令不能随便打断等规则。

模型侧主要使用 GPT 5.4/5.5 级别的 coding/reasoning agent 进行 vibe coding。这里的“vibe”并不等于没有纪律，恰恰相反：人的路线判断负责“往哪里走”，agent 负责“怎么把代码走出来”，而 verifier、regression、runtime test 和 full suite 负责判断改动是否可接受。与传统的手写代码为主、工具辅助检查不同，本项目更强调目标设定、实现自动化和验证闭环之间的分工：agent 可以快速生成和修改实现，但最终状态必须由可重复的测试与诊断结果确认。

8.2 测试矩阵

项目测试不是单一入口，而是覆盖多个层次：

- lexer/parser/semantic regression：锁前端语法和语义边界；
- IR/lower IR regression：锁 lowering 形状和 verifier contract；
- ValueSSA/MemorySSA tests：锁 SSA conversion、analysis、pass、allocator；
- machine tests：锁 machine lowering、select、layout、emit、encode、object/runtime/observe；
- compiler driver tests：锁 CLI、模式和输出行为；
- extension runtime tests：通过 RISC-V 汇编、链接和 qemu 实际运行扩展语言 witness。

每次修复真实 bug 后，都会补对应 regression。例如函数值实参中 `getint()` 被重复求值的问题，就通过 runtime witness 锁定：同一个普通实参必须只求值一次，然后根据 runtime tag 选择正确 callable 分支。

8.3 Verifier 与 dump

项目大量使用 verifier 和 dump expectation。Verifier 负责检查结构契约, dump 则让 IR/SSA/machine artifact 可观察。它们的价值在于: 很多 bug 不是最终程序崩溃才暴露, 而是在中间层就能被定位。

这也形成了一个开发经验: 复杂 feature 应避免依赖黑箱式成功。如果中间表示不可观察, 修复往往只能依赖最终现象反推原因, 效率低且容易误判。

8.4 全量回归

项目中一个重要纪律是: 阶段性改动必须经过 focused test 和 broad test 双重验证。典型顺序是:

1. 先运行最小 witness;
2. 再补 runtime 或 regression test;
3. 再跑相关子套件;
4. 最后跑 `make test` 全量测试。

这种流程使得 vibe coding 不至于变成仅凭直觉推进实现: 实现可以快速迭代, 但是否接受改动必须由测试结果决定。

9 经验教训

9.1 分层比局部技巧更重要

很多复杂问题不是靠某一个局部条件分支解决的, 而是靠正确分层降低复杂度。canonical IR、lower IR、ValueSSA、MemorySSA、machine backend 的边界越清晰, 后续扩展和调试就越容易。

9.2 语言扩展必须保守打开

函数值和闭包的实现尤其说明这一点。表面上只是“把函数当参数传”, 但很快会牵出 local alias、runtime tag、returned payload、closure env、callable-object IR、exactly-once evaluation 等一整串问题。如果过早宣称完整一等函数, 后续实现和验证成本会迅速上升。项目选择了保守 slice: 能支持的 family 稳定支持, 不能支持的 family 明确 gate。

9.3 性能优化必须讲证据

-perf 线也带来一个教训: 优化不能只看静态代码形状。某个 transform 是否值得保留, 要看真实热点、runtime A/B、compile-time 成本和正确性风险。有些优化在一个 witness 上效果很好, 在另一个 witness 上却可能带来负收益。

9.4 测试不是收尾，是设计的一部分

项目越往后，测试越不是实现后的附属品，而是 feature contract 的表达方式。很多“当前到底支持什么”的答案，不只在代码里，也在测试矩阵里。

10 总结

本项目最终形成了一个三模式编译器框架：`-riscv` 负责稳定基础编译与 RISC-V 输出，`-perf` 负责中端优化与 family-aware allocator 实验，`-extension` 负责语言特性扩展与 callable-object IR 探索。三个模式共享分层 IR、SSA、后端和测试基础设施，但各自有清晰边界。

从课程项目角度看，它的主要价值不只在“实现了多少功能”，更在于形成了一套可以继续扩展的工程方法：先分层，再验证；先保守支持，再逐步推广；先让中间状态可观察，再谈优化和复杂语义。本项目的核心经验是：面对复杂编译器行为时，不应急于扩大功能表面，而应优先建立 verifier、测试矩阵和可观察的中间层契约。

PS

1. 想要做好这个真的好难，最花时间的一集。
2. 已经尽力在让报告写得像人话了！但是真的很难表达明白 www。
3. OJ 真的什么报错都不给啊喂！只能对着几十万行的项目发呆 debug 是人类能战胜的嘛……
(lv8 hanoi2.sy 的 RE 陪伴到我做完，还是不知道为什么会 RE。)
4. vibe 有风险，管理需谨慎。一晚上 AI 写的代码能让我彻底不认识这个项目。

A Showcase 源程序与复现命令

A.1 主 showcase 源程序

```
struct Box { pair p; int bias; };

int sum_box(struct Box b) {
    return b.p.first + b.p.second + b.bias;
}

struct Box mkbox(int a, int b, int c) {
    struct Box box = {{a, b}, c};
    return box;
}

int apply(int f(int), int x) { return f(x); }
int pass(int h(int f(int), int x), int f(int), int x) {
    return h(f, x);
}

int make(int base)(int) {
```

```

    return closure [base] int (int y) { return base + y; };
}
int main() {
    struct Box b = mkbox(1, 2, 3);
    int f(int) = make(sum_box(b));
    return pass(apply, f, 4);
}

```

A.2 defer showcase 源程序

```

int main() {
    int x = 1;
    defer { x = x + 10; }
    capdefer(y = x) putint(y);
    x = 2;
    fndeferr putint(x);
    return x;
}

```

A.3 复现命令

```

build/tools/dump_middle_stage --extension ir showcase.sy
build/tools/dump_middle_stage --extension lower-ir showcase.sy
build/compiler -extension showcase.sy -o showcase.s

build/tools/dump_middle_stage --extension ir defer_showcase.sy
build/compiler -extension defer_showcase.sy -o defer_showcase.s

```

正文中的 dump 片段即由上述命令生成。报告正文只保留关键片段，是为了突出设计点并控制篇幅。

B 项目规模统计

下表按源码仓库中的主要目录统计行数。统计口径为排除以下目录后的文本行数：

```

build/
asanbuild/
third_party/
tmp/

```

lesson/ 也计入项目产物，因为它承担了设计复盘、代码阅读和后续开发记忆的角色。

部分	行数
src/	235,217
include/	17,939
src + include	253,156
tests/	238,529
docs/	40,349
lesson/	83,552
reports/	4,983
tools/	7,034
项目总量	629,432

这组数字不用于证明“代码越多越好”。它更适合说明项目的工程重心：实现与测试规模基本同量级，而 docs/ 与 lesson/ 也占了相当比例。对一个快速迭代的编译器项目来说，这些文字产物不是装饰，而是防止后续 agent 和人类读者迷路的地图。

C 运行与测试工具链

本附录给出从“写一个小程序”到“跑完整测试矩阵”的常用命令。以下命令默认在仓库根目录执行。

C.1 构建编译器

```
make
make compiler
make tools
```

默认 make 只构建 build/compiler，不会自动跑全量测试；这是为了兼容课程工具链中“先 build 编译器，再由外部评测运行”的习惯。make tools 会构建 dump/profile/allocator 诊断工具。

C.2 编译一个源程序

```
build/compiler -riscv input.sy -o output.s
build/compiler -perf input.sy -o output.s
build/compiler -extension input.sy -o output.s

build/compiler --enforce-all-paths-return-check -riscv input.sy -o output.s
```

三种模式分别对应基础 RISC-V 输出、性能优化路径和语言扩展路径。strict return gate 可以叠加在任意模式前，用来开启第 7 章介绍的 all-paths-return 审计。

C.3 从汇编链接并运行

生成的 .s 可以手动通过 RISC-V 工具链链接运行：

```
clang output.s -c -o output.o \
  -target riscv32-unknown-linux-elf -march=rv32im -mabi=ilp32

ld.lld output.o -L/opt/lib/riscv32 -lsysy -o output
qemu-riscv32-static ./output
```

仓库也提供了 make compile-one 与 make run-one 封装这条链路：

```
make compile-one SRC=input.sy MODE=riscv
make run-one SRC=input.sy MODE=riscv
make run-one SRC=input.sy MODE=extension IN=input.txt RUNOUT=stdout.txt
make run-one SRC=input.sy MODE=perf RETURN_CHECK=1
```

C.4 dump 中间层

调试编译器时，最常用的是把不同阶段 dump 出来比较：

```
build/tools/dump_middle_stage --extension ir input.sy
build/tools/dump_middle_stage --extension lower-ir input.sy
build/tools/dump_middle_stage --extension value-ssa-default input.sy

build/tools/dump_machine_stage machine-ir input.sy
build/tools/dump_machine_stage select input.sy
build/tools/dump_machine_stage emit input.sy

build/tools/dump_alloc_stage initial input.sy
build/tools/dump_alloc_stage final input.sy
```

dump_middle_stage 主要看 IR/lower IR/SSA；dump_machine_stage 主要看后端 machine-facing artifact；dump_alloc_stage 用于观察 allocator 计划和最终分配结果。

C.5 测试矩阵

全量测试入口是：

```
make test
make check
```

常用分项测试包括：

```
make test-lexer-regression
make test-parser-regression
make test-semantic-regression
make test-ir-regression
make test-lower-ir-regression
```

```
make test-value-ssa-regression
make test-value-ssa-alloc
make test-memory-ssa-pass
make test-machine-ir
make test-machine-select
make test-compiler-driver
make test-extension-runtime
```

此外还有更严格的构建/检查模式：

```
make test-fanalyzer
make test-asan
make test-strict-warnings
```

C.6 extension runtime 与 SysY suite

`make test-extension-runtime` 会调用：

```
tools/test_extension_runtime.sh
```

这条脚本会把 `-extension witness` 编译成 RISC-V 汇编，再用下面的工具真正运行，检查 stdout 和 exit status。

```
clang
ld.lld
qemu-riscv32-static
```

对于外部 SysY 测试集，可以使用 `suite sweep` 工具：

```
python3 tools/sweep_sysy_suite.py path/to/suite --compiler build/compiler
```

如果测试集带真实输入文件，应使用真实输入运行；空 stdin 的 qemu 结果不能作为性能或正确性结论。

C.7 性能与诊断工具

```
python3 tools/profile_compile_pipeline.py input.sy
build/tools/profile_compiler_stages input.sy
build/tools/profile_backend_layers input.sy
build/tools/diag_allocator_stages input.sy

python3 tools/analyze_riscv_asm.py output.s
python3 tools/classify_machine_select_hotspots.py dump.txt
```

`-perf` 相关结论应优先使用真实 runtime A/B，而不是只看静态指令数或“汇编看起来更短”。这些诊断工具主要用于定位热点、解释形状和缩小问题范围。

C.8 报告编译

本报告使用 XeLaTeX 编译：

```
latexmk -xelatex -interaction=nonstopmode \  
-halt-on-error -outdir=reports reports/course_report.tex  
  
latexmk -c -outdir=reports reports/course_report.tex
```

D 进一步阅读路线

报告正文刻意只展开了最有代表性的设计点。若希望继续核对实现细节，可以按下面路线阅读源码、测试、设计文档和 lesson。每组先给阅读目的，再给入口文件；这些不是完整目录树，只是适合继续顺藤摸瓜的起点。

D.1 项目总览与工作记忆

先看这些文件，可以了解 agent 如何启动、项目当前处在什么阶段，以及哪些工程事实已经被记录为工作记忆。

```
AGENTS.md  
docs/NEXT_STEPS.md  
docs/ENGINEERING_MEMORY.md  
lesson/README.md
```

D.2 前端、语义与 strict return

这一组用于补足词法、语法、语义和 all-paths-return 的细节，其中控制流摘要文件对应正文第 7 章。

```
lesson/core/lexer_lesson.md  
lesson/core/parser_lesson.md  
lesson/core/semantic_lesson.md  
src/parser/  
src/semantic/  
src/semantic/semantic_core_flow.inc  
tests/semantic/semantic_regression_scope_cf.inc  
tests/semantic/semantic_regression_callable_flow.inc  
tests/compiler/compiler_driver_test.c
```

D.3 IR、lower IR 与可观察 dump

这一组对应正文里的 canonical IR、lower IR、verifier 和 dump。想核对 showcase 片段，可以从 dump_middle_stage 重新生成。

```
lesson/core/ir_lesson.md
lesson/core/lower_ir_lesson.md
docs/ir/LOWER_IR_DESIGN.md
include/ir.h
src/ir/
tests/ir/
include/lower_ir.h
src/lower_ir/
tests/lower_ir/
build/tools/dump_middle_stage
```

D.4 SSA、MemorySSA 与优化

这一组适合继续看 ValueSSA、MemorySSA、优化 pass 和 `-perf` 工作流。正文只讲了主线，详细 pass 取舍和 A/B 记录主要在优化计划里。

```
lesson/ssa/value_ssa_lesson.md
lesson/ssa/memory_ssa_lesson.md
docs/ssa/VALUE_SSA_DESIGN.md
docs/ssa/MEMORY_SSA_DESIGN.md
src/value_ssa/
src/value_ssa_pass/
tests/value_ssa/
src/memory_ssa/
tests/memory_ssa/
docs/OPTIMIZATION_PLAN.md
docs/GENERAL_OPT_PASSES_TODO.md
```

D.5 family-aware allocator

正文的 allocator 例子只展示了关键字段。若要继续看 family、priority、targeted eviction 和 result shaping，可以从这些入口读。

```
lesson/ssa/value_ssa_alloc_lesson.md
docs/ssa/VALUE_SSA_ALLOCATOR_PLAN.md
docs/ssa/VALUE_SSA_ALLOCATOR_RESULT_SHAPING_PLAN.md
src/value_ssa_alloc/
tests/value_ssa/
```

D.6 后端与 RISC-V 输出

这一组对应 `-riscv` 与 machine backend。machine 层文件很多，建议先从 plan 和 lowering 主线读起，再看 object/runtime/observe。

```
lesson/machine/README.md
docs/backend/MACHINE_IR_PLAN.md
docs/backend/MACHINE_SELECT_PLAN.md
docs/backend/MACHINE_EMIT_PLAN.md
docs/backend/MACHINE_ENCODE_PLAN.md
include/machine/
src/machine/lowering/
tests/machine/lowering/
src/machine/object/
src/machine/runtime/
src/machine/observe/
```

D.7 -extension 语言扩展

这一组对应正文第 5 章和第 6 章。建议先读总览，再按 defer、aggregate、function value、closure、callable-object IR 分组深入。

```
lesson/language/defer_family_lesson.md
docs/language/DEFER_FEATURE_PLAN.md
docs/language/FUNCTION_EXIT_DEFER_PLAN.md
docs/language/DEFER_CAPTURE_PLAN.md

lesson/language/aggregate_lesson.md
lesson/language/aggregate_boundary_lesson.md
docs/language/PAIR_FEATURE_PLAN.md
docs/language/STRUCT_FEATURE_PLAN.md
docs/language/AGGREGATE_PARAM_RETURN_PLAN.md

lesson/language/function_values_lesson.md
lesson/language/function_implementation_map_lesson.md
lesson/language/function_supported_code_lesson.md

lesson/language/closure_object_lesson.md
lesson/language/returned_callable_lesson.md
lesson/language/returned_closure_transport_lesson.md
lesson/language/higher_order_callable_lesson.md

lesson/language/callable_object_ir_lesson.md
docs/ir/CALLABLE_OBJECT_IR_DESIGN.md
docs/language/CALLABLE_OBJECT_IR_PLAN.md
```

扩展语言最密集的功能和 gate 测试入口在：

```
tests/compiler/compiler_driver_test.c
tests/semantic/semantic_regression_test.c
```

D.8 测试体系

最后，如果要从测试角度理解“项目到底锁住了哪些行为”，可以从这些入口开始。

```
lesson/core/tests_lesson.md
tests/lexer/
tests/parser/
tests/semantic/
tests/ir/
tests/lower_ir/
tests/value_ssa/
tests/memory_ssa/
tests/machine/
tests/compiler/
```